02: Sampling

Stat 120 | Spring 2026

Prof Amanda Luby

Press the "Run Code" button below to run the chunk of code. This (1) loads the libraries that we need and (2) tells R to read `mission_data.csv` from my website folder into the R session and call it `mission_data`.

```
library(dplyr)
mission_data = read.csv("https://aluby.github.io/stat120/data/mission_data.csv")
```

> **Note**
>
> You should notice that R did not appear to do anything when you ran the above code. That's because everything was done in the background and R will only show you what is requested. So don't panic if you don't see any output!

Throughout the term I'll refer to small pieces of code as "code chunks." This is both descriptive and will be what RStudio (our interface) calls them later on. For today, we'll focus on running small code chunks, and on Monday we'll get practice writing our own.

We can get an overview of the dataset by printing the first few rows with the `head()` command:

```
head(mission_data)
```

Another function, `glimpse()`, which lives in the {tidyverse} package, gives us another view of the dataset:

```
glimpse(mission_data)
```

1 Checkpoint questions

- How many rows are in the `mission_data` dataset?
- What does each row represent?
- How many variables are in the dataset? Are any categorical? Quantitative?

Instead of relying on our brains to do a random sample, we can use R to ensure that we get a random sample!

The code below selects a random sample of word positions to use

```
set.seed(091824) # Sets the random seed so we all get the same answer
sample = sample(1:364, size = 10) # selects a random sample of size 10 from the numbers 1-364
sample # prints the random sample of numbers
```

The next chunk of code `slices` our population to draw our sample. Note that the `position` variable should match the `sample` output above.

```
mission_sample = slice(mission_data, sample)
```

By running the name of the dataset, we print the entire thing:

```
mission_sample
```

R has built-in functions to compute summary statistics (like mean, median, and standard deviation) for a specific column. To "extract" a variable from the dataset, we use `$`. For example, we can extract the `length` variable from the `mission_sample` dataset and calculate the mean:

```
mean(mission_sample$length)
```

2 Checkpoint question

To try other random samples, change (or remove!) the `set.seed()` line of code below, and try re-running the rest of the code. Do you ever get a sample mean that looks like your "by hand" sample mean?

```
set.seed(091824)
sample = sample(1:nrow(mission_data), size = 10)
mission_sample = slice(mission_data, sample)
mean(mission_sample$length)
```

Finally, let's see what the population mean is!

3 Checkpoint question

Use the code box below to compute the mean of the `length` variable in the original `mission_data` dataset. How close was your "by-hand" sample? How close were your true random samples?

```
#| min-lines: 5
```

4 Function quick reference

The following table summarizes the functions we learned today:

Function	Purpose
`read.csv("file_path_or_URL")`	Read a CSV data set from a file or the web
`head(data)`	Print the first few rows of a data set
`glimpse(data)`	Get a quick overview of a data set (from {dplyr} package)
`mean(x)`	Calculate the mean of a quantitative variable
`sample(x, size = n)`	Selects a random sample from `x` of size `n`
`slice(data, rows)`	Selects specific rows from a data set